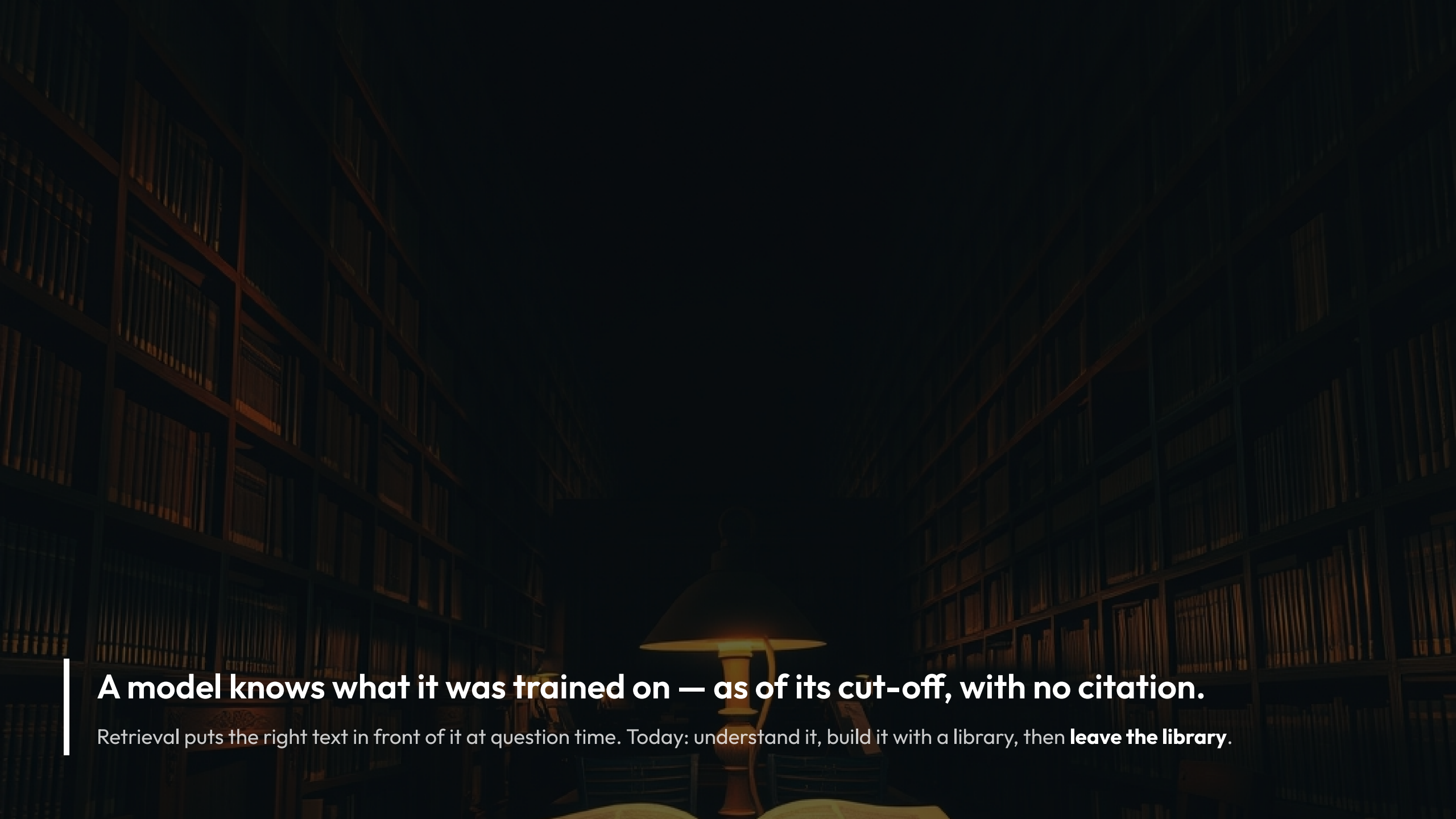
RAG: Basics, Practice, and Custom Retrieval

LLM-Agents

Presented by: **Aleksandr Algazinov**



HSE University



A model knows what it was trained on — as of its cut-off, with no citation.

Retrieval puts the right text in front of it at question time. Today: understand it, build it with a library, then **leave the library**.

Motivation

WHAT THE MODEL HAS

Its training data, frozen at the cut-off. No sources, no way to ask "where did that come from?"

WHAT IT DOES NOT HAVE

Your company's documents. Last week's changes. Anything you would need to cite.

RETRIEVAL

Find the right text at question time and put it in the prompt — with its source.

1 **Understand retrieval** — chunking, embeddings, indexes, hybrid search

2 **Build it with a library** — LlamaIndex, live, in a local web app

3 **Leave the library** — rerankers, hard negatives, metrics, serving levers. The reason this lecture exists

Already in production: DoorDash

THE SYSTEM

- Dasher-support chatbot: "a RAG system that retrieves information from knowledge base articles"
- "Each day, this system autonomously assists thousands of Dashers"

ITS FIRST PROBLEM WAS THE QUERY, NOT THE INDEX

- In a multi-turn chat, "we must summarize the issue from that dialogue"
- Because "the summary's presentation affects the result produced by the retrieval system"
- How DoorDash checks the *answers* — guardrail and judge — is Lecture 13's

> **What you retrieve depends on what you ask with.**
>
> Jia et al. · DoorDash Engineering · 2024 (company-reported)

DISCUSSION

Models take a million tokens now.

Why not paste the whole knowledge base into every prompt? Is RAG obsolete?

Long context does not retire retrieval

COST AND LATENCY, PER QUERY

You pay for — and wait on — every input token, on every question.

POSITION MATTERS

Accuracy is often highest when the fact sits at the start or end of the context, and drops in the middle — *Lost in the Middle*.

FRESHNESS AND ACCESS CONTROL

A pasted corpus is stale and the same for every user. Retrieval filters per user and per update.

THEY COMPOSE

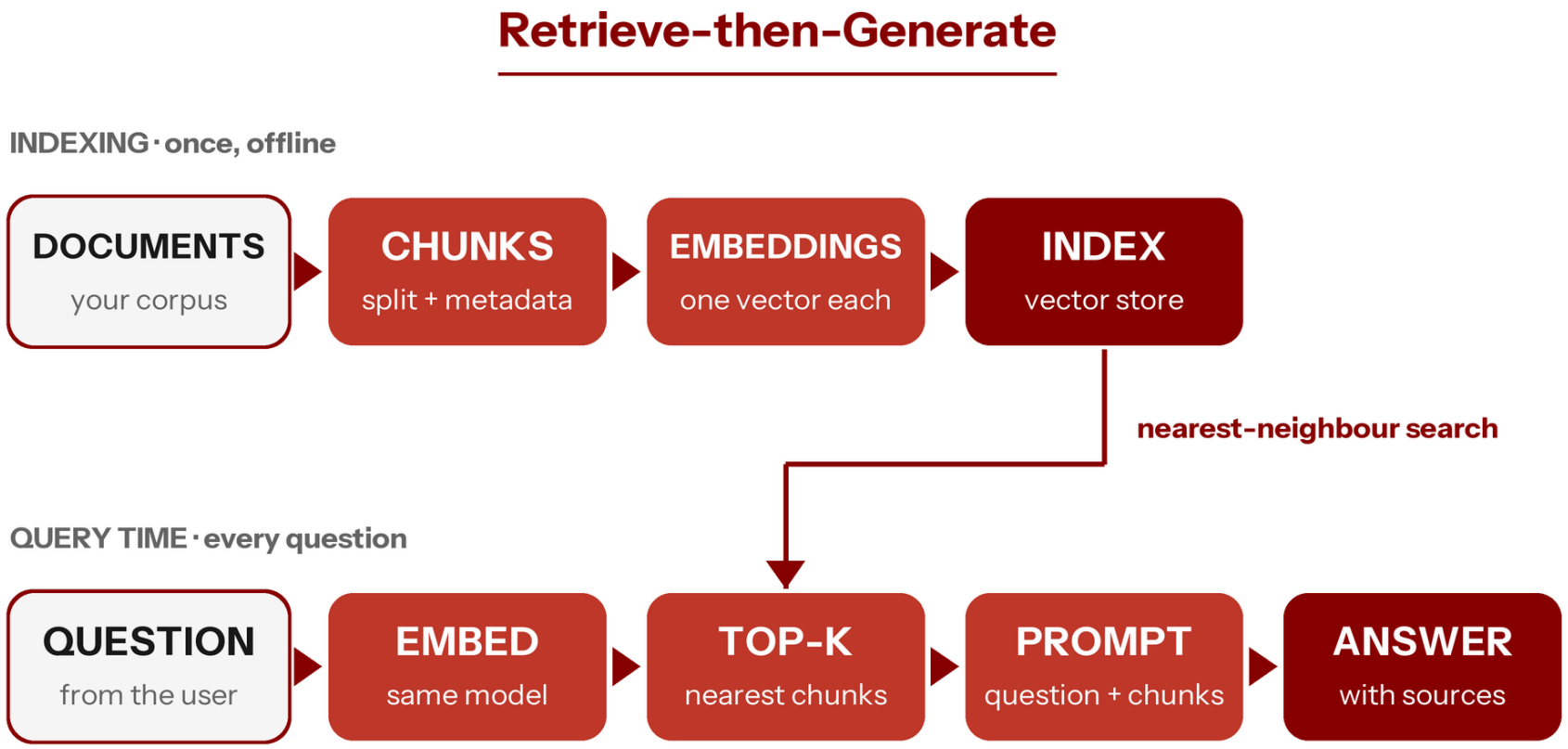
Retrieve *then* use the window well. How long contexts degrade more broadly is Lecture 07's.

Fundamentals

The loop, vectors, indexes

The retrieve-then-generate loop

Two phases: build the index once, then retrieve for every question.



Embeddings, applied to retrieval

LECTURE 02: WHAT AN EMBEDDING IS

- A vector that represents meaning
- Similar meaning → nearby vectors
- Similarity by dot product / cosine

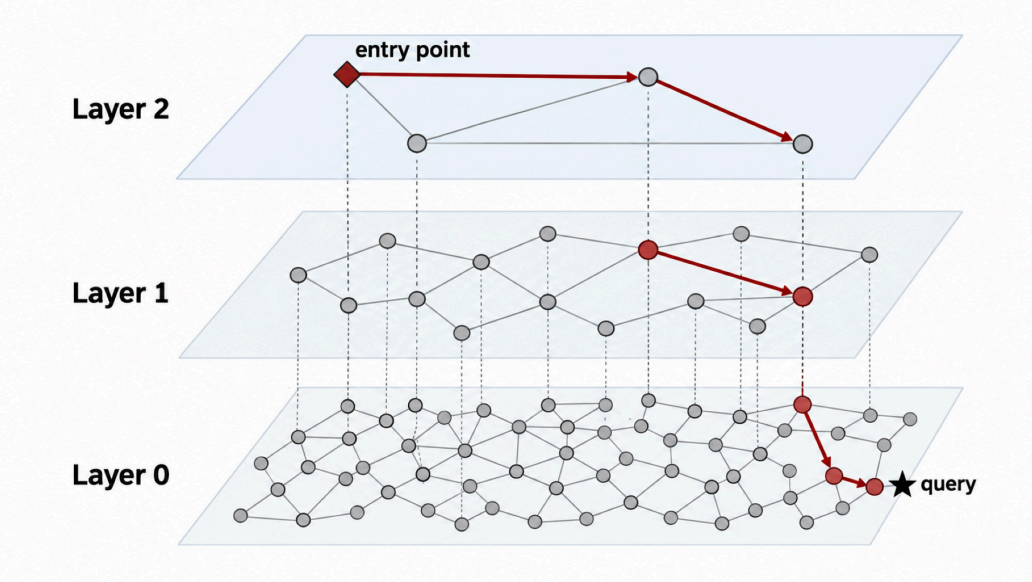
TODAY: USING IT TO SEARCH

- **One vector per chunk**, stored in an index
- Embed the question, find the nearest chunks
- The query and the chunks **must be embedded by the same model** — two models, two unrelated spaces

> **Retrieval is nearest-neighbour search over your chunks.**
>
> The rest of this block is about doing that search fast enough.

Exact vs approximate nearest neighbour

- **Exact search** scans every vector — linear in the corpus
- **ANN** trades a little recall for a lot of speed
- **HNSW**: greedy hops on sparse upper layers, refine on the dense bottom — search scales **logarithmically**
- **IVF**: partition into cells, search a few
- Speed vs memory vs accuracy (Faiss)



Where the loop breaks

1 · WRONG CHUNK RETRIEVED

Close in vector space, wrong in meaning. The model answers confidently from it.

2 · RIGHT CHUNK NOT RETRIEVED

It exists — but it is not in the top-k. The model never sees it.

3 · RIGHT CHUNK, LOST AMONG OTHERS

Retrieved, but buried among distractors in the prompt.

THESE THREE SHAPES ORGANISE THE REST OF THE LECTURE

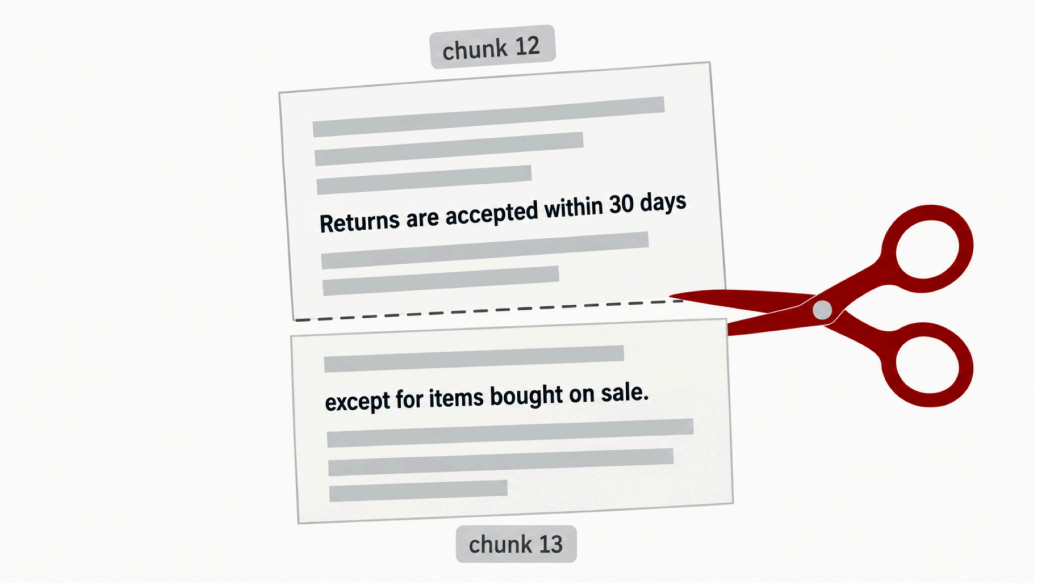
Seven Failure Points (Barnett et al. 2024) names these and more — missing content, missed top-ranked documents, not in context — and notes that "validation of a RAG system is only feasible during operation".

Chunking

Where to split — and what context to return

Chunking — and what it destroys

- **Fixed-size + overlap** — N-token windows; repeat boundary text
- **Recursive** — coarse separators first; finer ones if a piece is too long
- **Sentence / paragraph-aware** — keep natural boundaries
- **Structure-aware** — headings, sections, articles

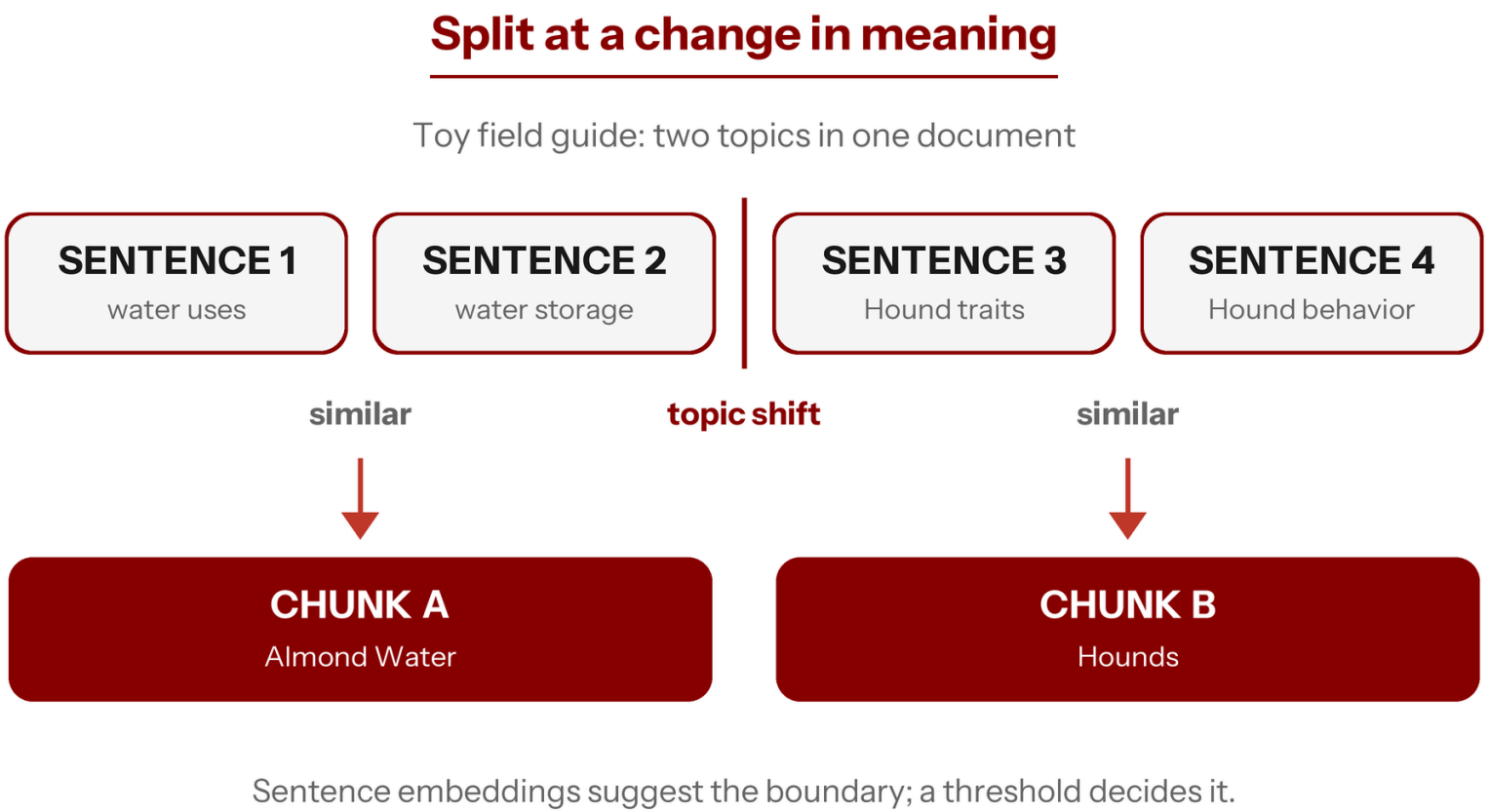


WHAT IT DESTROYS

Keep source and section metadata. A rule and its exception can still land in different chunks.

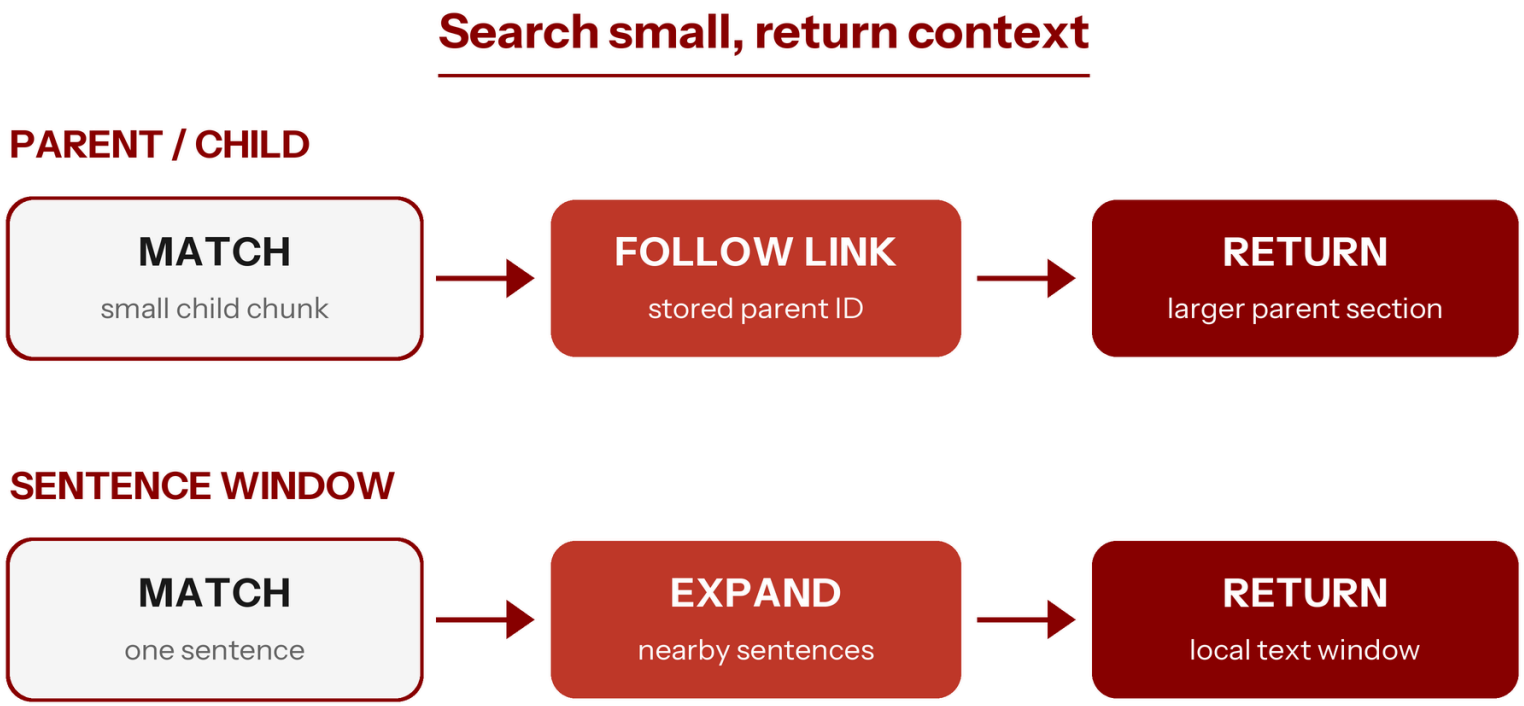
Semantic chunking: split when the topic changes

Split at changes in sentence-group embeddings. More ingestion work; thresholds need tuning.



Parent/child and sentence windows

Retrieve focused text, expand before generation. Parent links and neighboring sentences offer two ways to add context; both cost prompt tokens.



Expansion needs an explicit policy; parsing alone does not do it.

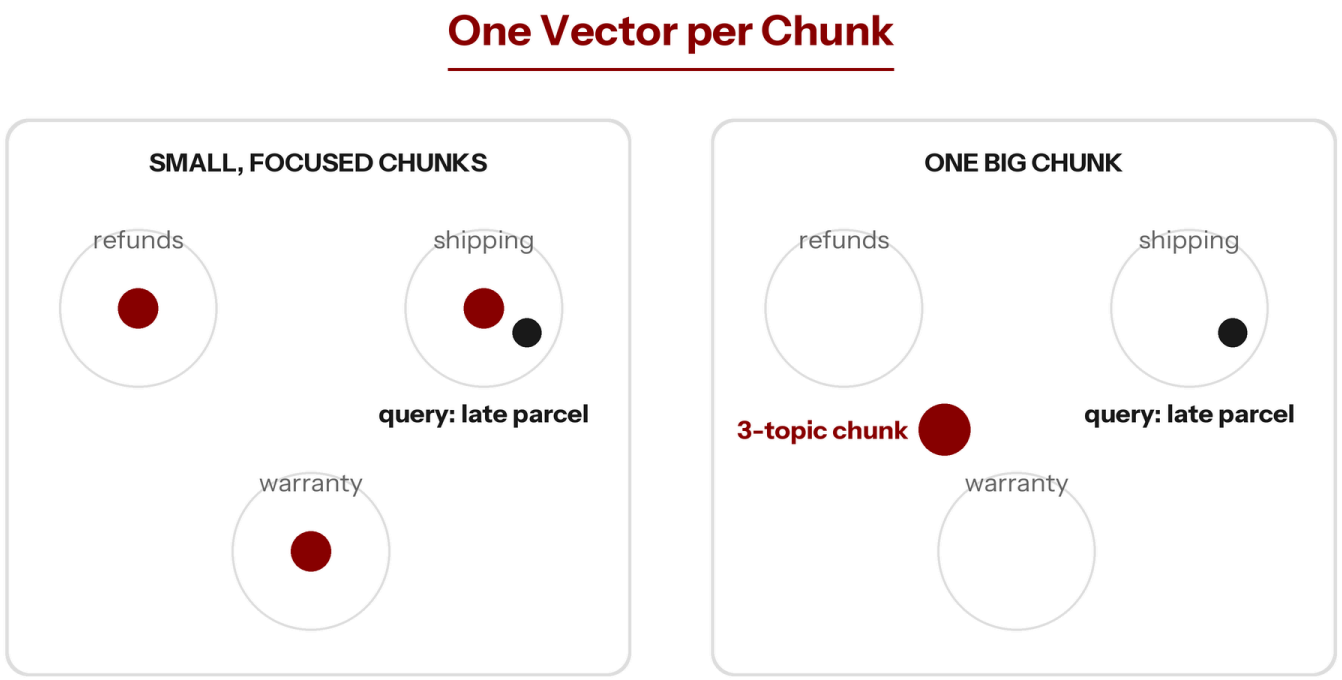
DISCUSSION

Small chunks lose context.

So just make chunks bigger — more context, better answers?

Bigger chunks dilute the signal

One embedding per chunk. Mixed topics can dilute the signal; larger chunks fit fewer passages into the prompt. Search focused text, then expand its context.



A chunk about three topics embeds between them, near none of them

The chunk lost its document

THE CHUNK

"The company's revenue grew by 3% over the previous quarter."

Which company? Which quarter? The embedding cannot know — and neither can BM25.

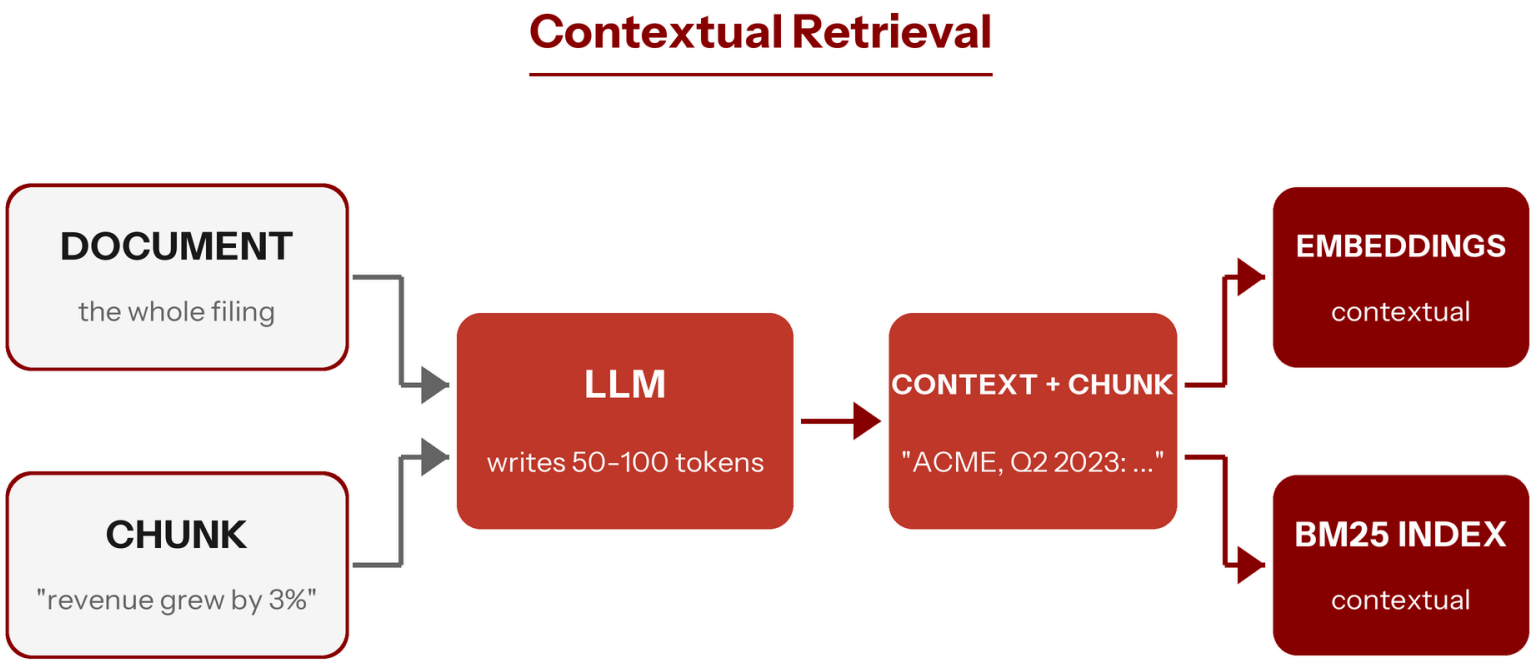
THE FIX: GIVE EACH CHUNK ITS CONTEXT BACK

- An LLM writes a short context — usually **50–100 tokens** — situating the chunk in its document
- Prepend it before embedding (*contextual embeddings*)
- ...and before building the BM25 index (*contextual BM25*)

> **This is contextual retrieval.**
>
> Anthropic, *Introducing Contextual Retrieval*, 2024

Contextual retrieval: method and result

Top-20 retrieval failures, vendor-reported: **−35%** with contextual embeddings · **−49%** adding contextual BM25 · **−67%** adding a reranker.



One LLM call per chunk, paid once at indexing time, not per question

DISCUSSION

A 3-page return-policy FAQ · a 400-page tax code with numbered articles · a support-chat export.

Which chunking fits each — and where does fixed-size chunking break?

Three corpora, three chunkings

3-PAGE FAQ → DO NOT CHUNK

- **Fits:** the whole FAQ in the prompt. Under ~200K tokens: "no need for RAG" (Anthropic); caching makes it cheap
- **Breaks:** a window splits "within 30 days" from "except sale items"

400-PAGE TAX CODE → PER ARTICLE

- **Fits:** one chunk per article, its path as metadata: `Part II › Ch. 23 › Art. 217 › §3`
- **Breaks:** a window straddles two articles — Art. 217 quoted under Art. 218's number

CHAT EXPORT → PER CONVERSATION

- **Fits:** the thread is the unit; product, date, outcome as metadata
- **Breaks:** question cut from answer — and one customer's chat glued to the next: a privacy leak

> **Chunk by the corpus's own unit; size is the fallback.**

Hybrid search: two retrievers that fail differently

BM25 — SPARSE

Exact term overlap, weighted by how rare the term is. Finds `ERR_4012`. Misses "car" for "automobile".

DENSE — EMBEDDINGS

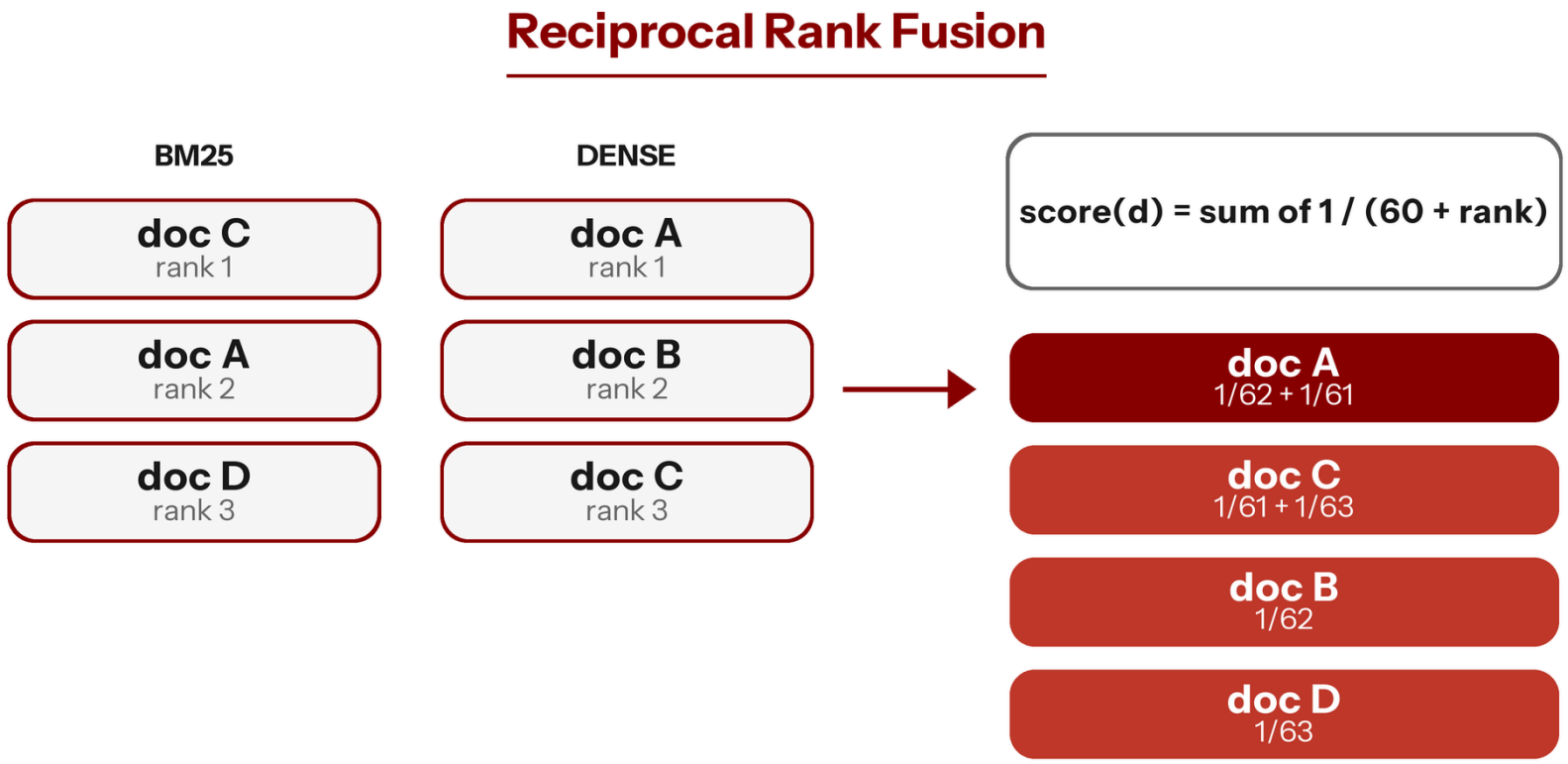
Meaning. Finds "automobile" for "car". Blurs a SKU into its neighbours.

Keep both — and fuse their ranks, not their scores.

A BM25 score and a cosine live on different scales; a rank is a rank.

Reciprocal rank fusion

Each document scores the sum of 1 / (60 + its rank) across the lists — no calibration needed.



Only ranks are fused, so a BM25 score and a cosine never have to be put on one scale

RRF, precisely

$$\mathrm{RRF}(d) \;=\; \sum_{r \in \text{rankers}} \frac{1}{k + \mathrm{rank}_r(d)}, \qquad k = 60$$

Cormack, Clarke & Büttcher 2009: "k = 60 was fixed during a pilot investigation" and found near-optimal

LLAMAINDEX SHIPS IT

`QueryFusionRetriever` — RRF "without excessive computation".

WATCH ONE KNOB

With `num_queries > 1` it asks an LLM to write extra queries — that **adds** latency.

DISCUSSION

> **Dense retrieval beats BM25 on benchmarks.**
>
> So drop BM25?

Keep the sparse index

WHERE DENSE IS WEAKEST, BM25 IS EXACT

- Identifiers, error codes, product SKUs
- Names and rare terms
- Contextual BM25 (earlier) is one more reason to keep a sparse index at all

AND OUT OF DOMAIN, DENSE STUMBLES

- BEIR, 18 datasets, zero-shot: "BM25 is a robust baseline"
- Dense models "often underperform" other approaches out of domain
- Hybrid + RRF keeps both

> **"Beats on the benchmark" is not "beats on your corpus".**

Build it

LlamaIndex, live

Live demo — The Backrooms Archive

OPEN LOCALHOST:8501

- **Documents:** browse 50 English articles — levels, entities, objects
- **Ask the archive:** ask a question and follow the numbered sources

TRY IT, THEN CHECK THE SOURCE

- What is Almond Water used for?
- Can travelers meet on Level 0?
- What did I eat for breakfast yesterday?

> **Does the cited passage actually support the answer?**

English Backrooms Wiki snapshot · CC BY-SA 3.0 · Docker + LlamaIndex

DISCUSSION

Why does the system we just built suck?

It can answer questions and show sources. What have we still not established about its quality, latency, or behavior on difficult questions?

A working demo leaves engineering questions open

QUALITY IS UNMEASURED

No labelled query set, no checked hard negatives. Can it distinguish a relevant passage from a similar but wrong one?

RANKING IS MINIMAL

Dense top-5 only. No BM25, reranker, or parent/window expansion. Important evidence may never reach the prompt.

LATENCY IS UNPROFILED

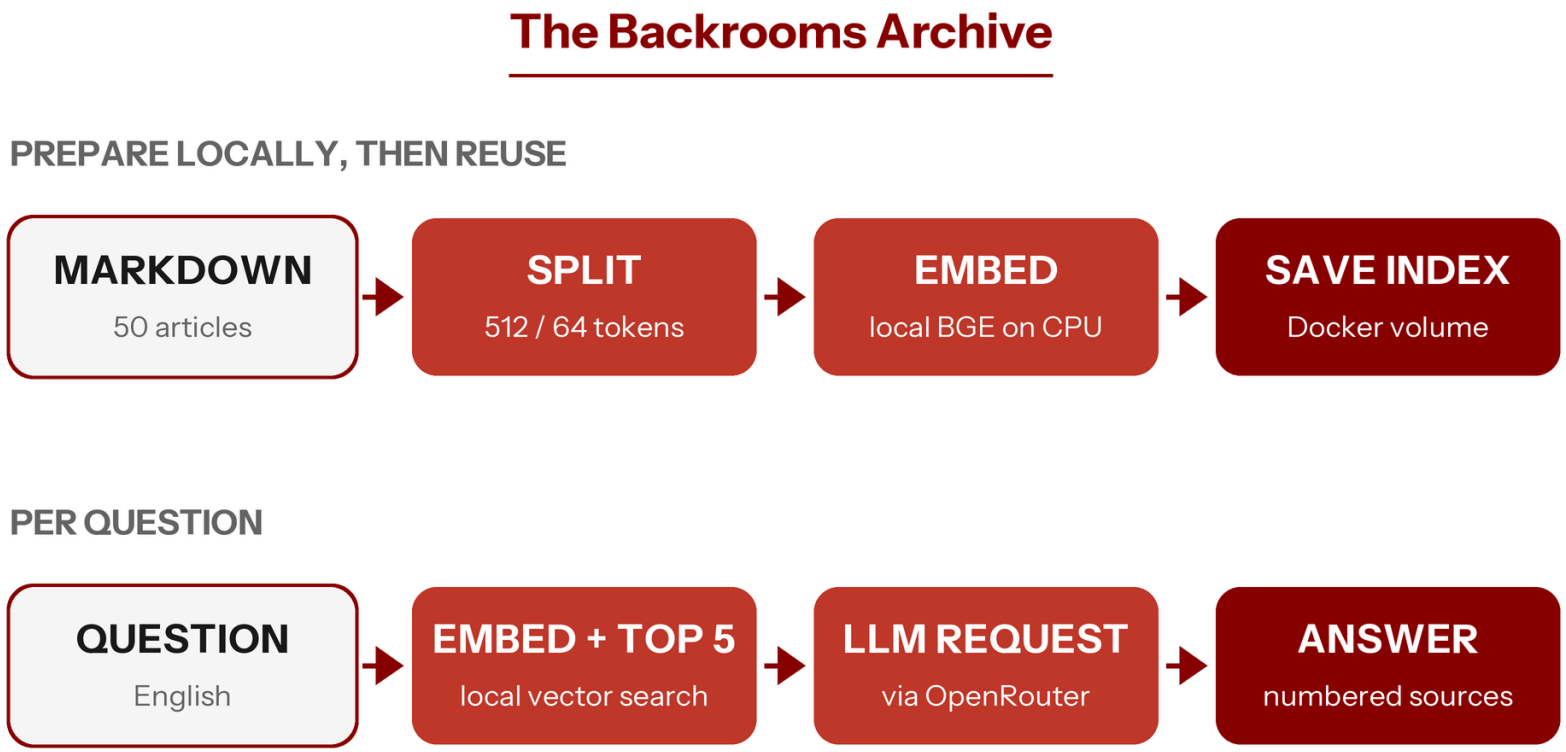
Local query embedding + search + one remote generation request. We have not timed the stages separately.

CONTROL NEEDS IMPLEMENTATION

A fixed retrieval policy; no per-user document filtering. Valid source numbers do not prove that every claim is supported.

What ran behind the UI

The browser is local; answer generation is remote. The saved index avoids re-embedding the corpus on every question.



One generation request per answered question; embeddings stay local.

The retrieval core in LlamaIndex

`llama-index-core==0.14.25` · `llama-index-embeddings-huggingface==0.8.0`

```python
from llama_index.core import VectorStoreIndex
from llama_index.core.node_parser import SentenceSplitter
from llama_index.embeddings.huggingface import HuggingFaceEmbedding

# docs: Markdown Document objects with titles and source URLs
embed_model = HuggingFaceEmbedding(
    model_name="BAAI/bge-small-en-v1.5", device="cpu")
splitter = SentenceSplitter(chunk_size=512, chunk_overlap=64)
index = VectorStoreIndex.from_documents(
    docs, embed_model=embed_model, transformations=[splitter])
retriever = index.as_retriever(similarity_top_k=5)
nodes = retriever.retrieve("What is Almond Water used for?")
```

The app numbers these passages, adds the question, then calls the configured LLM once. The prompt asks it to cite the passages or say the answer was not found.

Defaults are choices — inspect the actual component

Setting	LlamaIndex 0.14.25 default	Our demo
SentenceSplitter size / overlap	1024 / 200 tokens	512 / 64 tokens
Retrieved chunks	2	5
Embedding model	OpenAI fallback	Local BGE, 384 dimensions
Vector storage	In-memory by default	Loaded in memory, persisted on disk

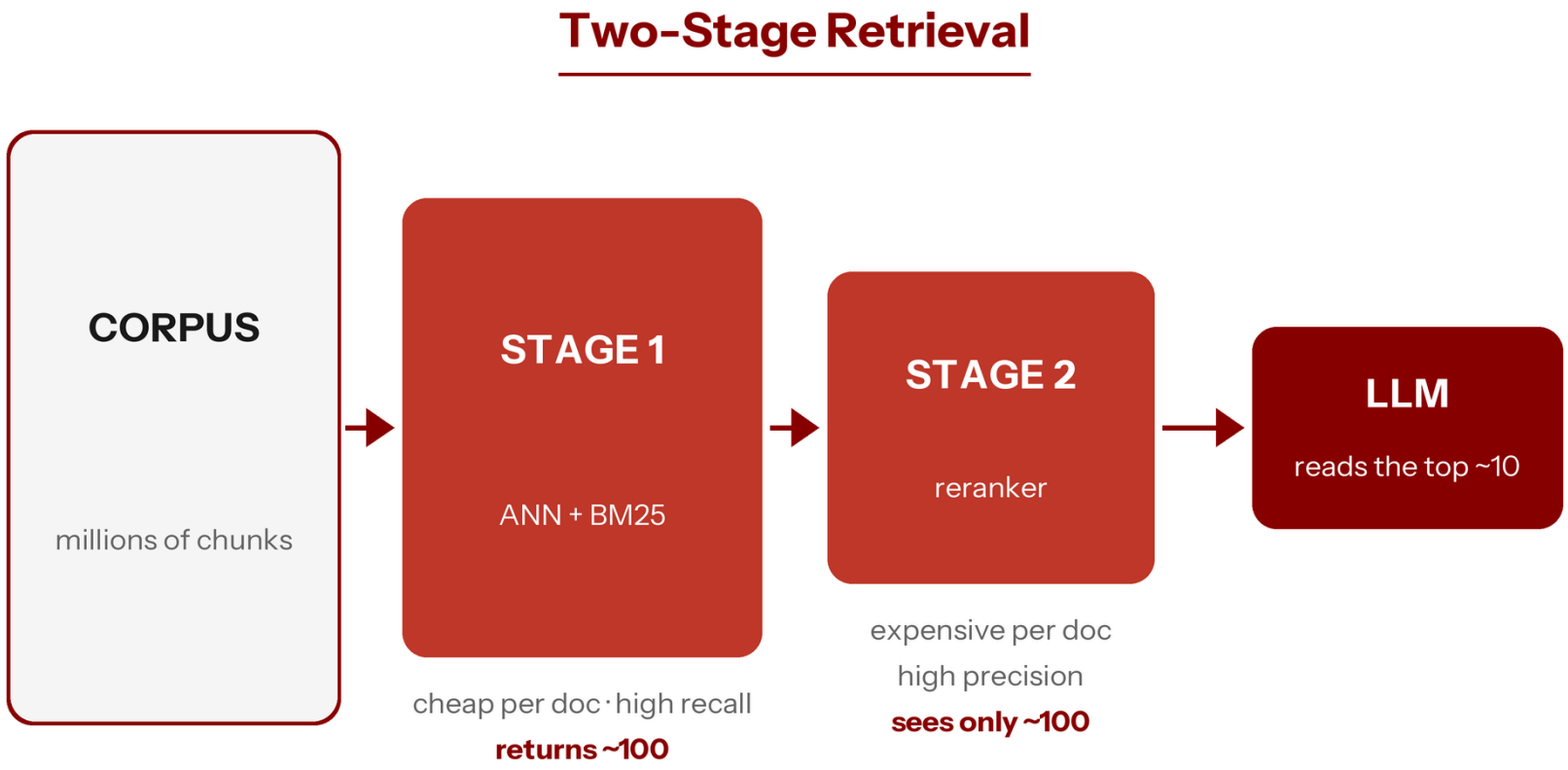
> **LlamaIndex lets us replace these components.**
>
> Explicit settings give us control. Labelled queries tell us whether those settings work.

Custom retrieval

Own the retrieval decisions

Two-stage retrieval

Cheap and high-recall first; expensive and high-precision only on what survives.



The expensive model's cost scales with how many documents it scores

The latency argument for the split

WHY NOT RERANK EVERYTHING?

- The expensive model's cost scales with **how many documents it scores**
- Scoring "thousands or millions of (query, document)-pairs would be rather slow" (Sentence-Transformers docs)
- So: retrieve ~100 with a bi-encoder, rerank those

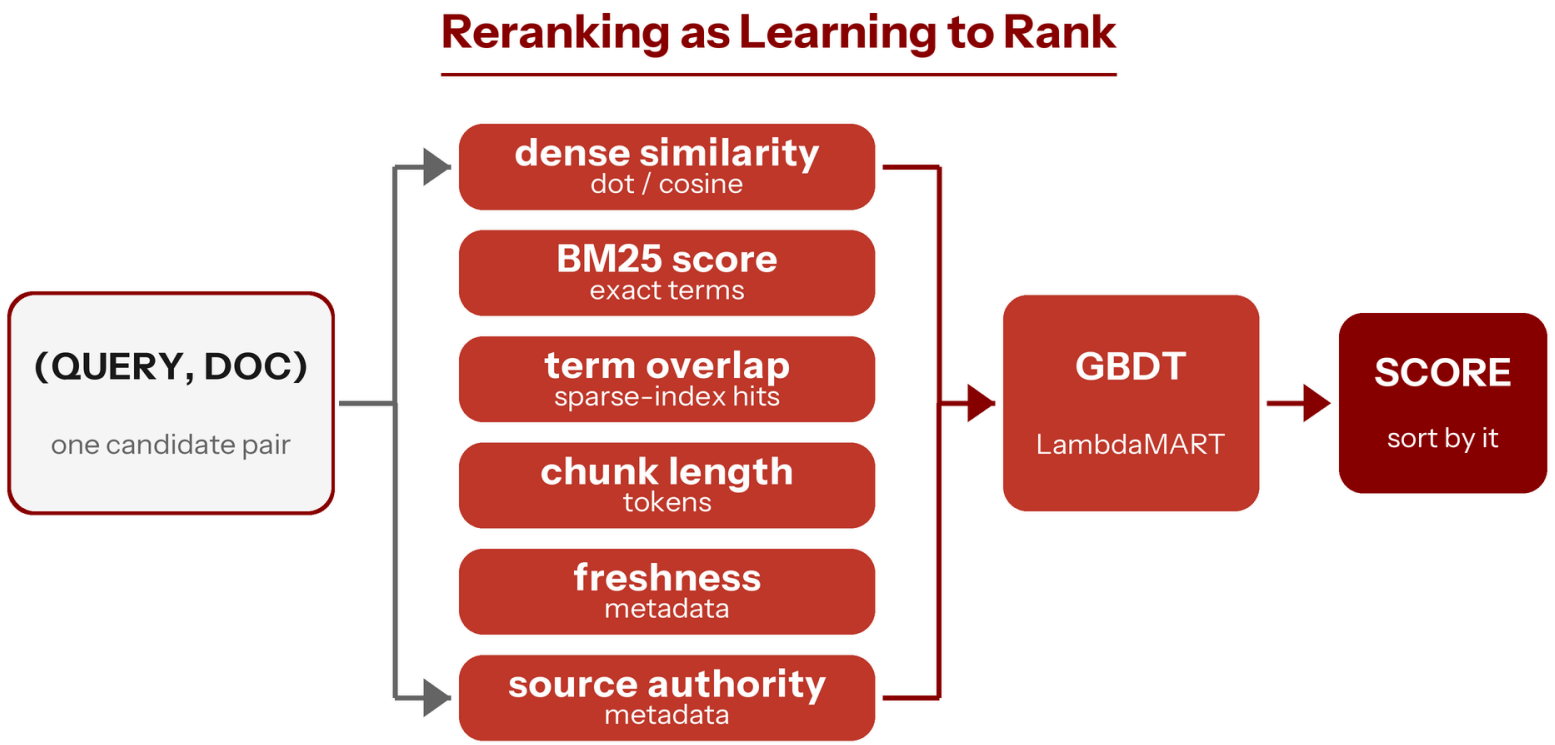
OUR DEMO STOPS AT STAGE ONE

- Embed, vector search, top-k — the demo you just saw
- Stage two is where the engineering lives: **what** scores the candidates, trained **on what**, measured **how**

> **Next: build stage two, measure it — then tune both stages.**

Reranking is learning to rank

Familiar ground: a **feature vector** per (query, document) pair, a **label**, a **ranking loss** — tabular ML applied to unfamiliar data.



A dozen numbers per pair: no neural forward pass at rerank time

The GBDT reranker

LAMBDAMART

Gradient-boosted trees with a ranking objective. "An ensemble of LambdaMART rankers won Track 1 of the 2010 Yahoo! Learning To Rank Challenge" (Burges 2010).

IN CODE

- LightGBM `lambdarank` — or `rank_xendcg`, "faster than and achieves the similar performance as lambdarank"
- CatBoost `YetiRank` — the default for `CatBoostRanker`

CHEAP

A tree ensemble over a dozen numbers — no neural forward pass per pair.

IN PRODUCTION

Airbnb search: "much of the initial gains were driven by a gradient boosted decision tree model".

The cross-encoder reranker — and ColBERT between

CROSS-ENCODER (MONOBERT)

- Query and document read **together** by a transformer
- Beat the previous MS MARCO state of the art by **27% in MRR@10**
- One full forward pass **per pair**

THROUGHPUT, ON THEIR BENCHMARK

- MiniLM-L6: ≈ **1,800** docs/s (NDCG@10 74.30)
- TinyBERT-L2: ≈ **9,000** docs/s (69.84)
- Hardware not stated

COLBERT — LATE INTERACTION

- "Two orders-of-magnitude faster" than BERT-based rankers
- "Four orders-of-magnitude fewer FLOPs per query"

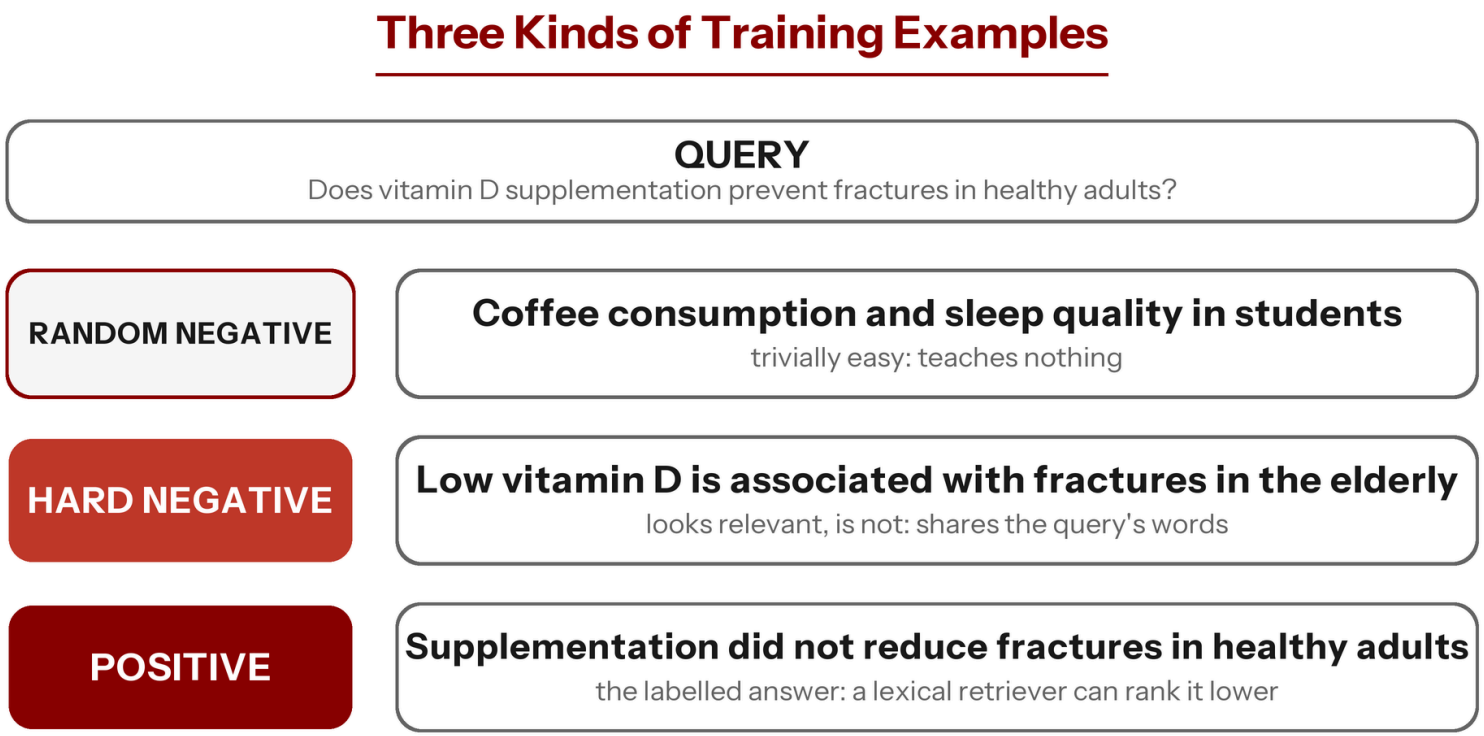
GBDT vs cross-encoder: ship both

	GBDT reranker	Cross-encoder
Cost per pair	A tree ensemble over numbers — cheap	A transformer forward pass — expensive
Needs	Engineered features	Nothing but raw text
Understands	What the features capture	Meaning
Where it sits	On ~100 candidates	On the top ~10

> **Not a choice — a cascade.**
>
> Cheap model on many, expensive model on few.

Training a ranker: which negatives?

A ranker learns from (query, positive, negatives). **Random** negatives are trivially easy — the model learns nothing it did not already know.



Illustrative example: a random negative is too easy to teach the ranker anything

DISCUSSION

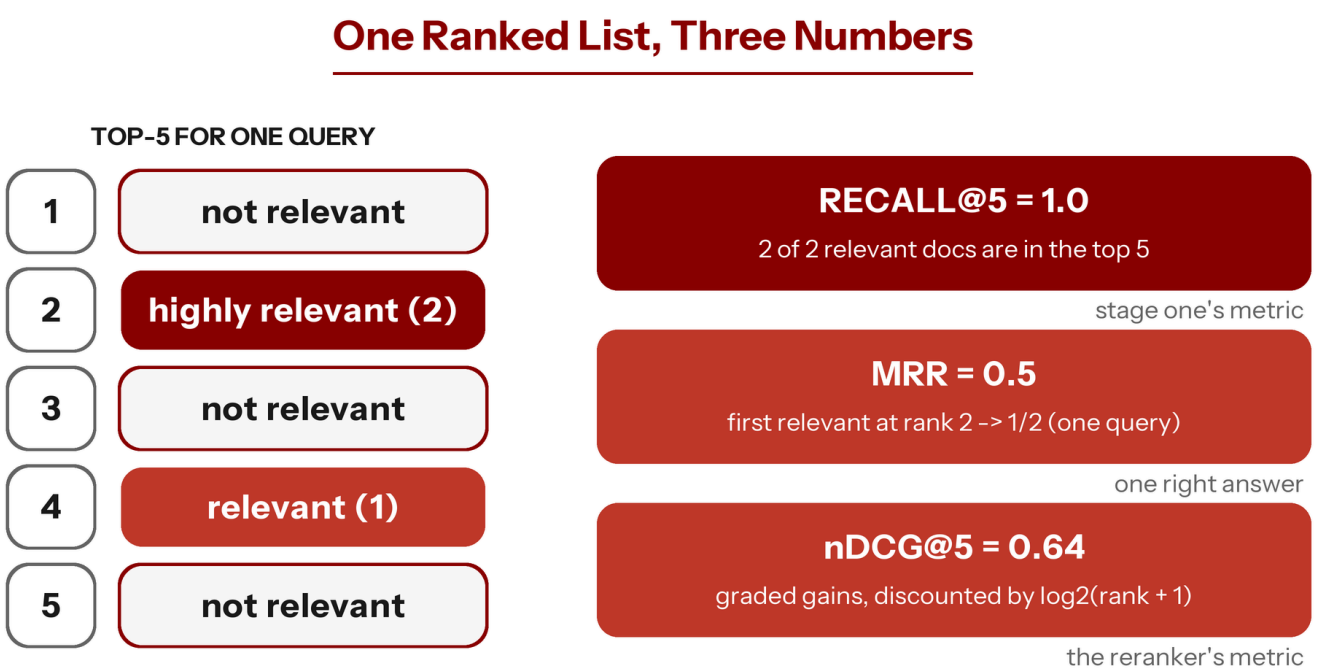
> **For negatives, just take the top-ranked documents that are not labelled relevant.**
>
> Those are the hardest negatives there are — right?

The hard-negatives trap

1. **Labels are sparse.** Only a few documents per query are labelled relevant — not every relevant one in the corpus.
2. **So "top-ranked but unlabelled" is often simply unlabelled.** RocketQA inspected such negatives: about **70%** were actually positives.
3. **Training on them teaches the model to push right answers away.**
4. **Fix: denoise.** Keep only candidates a cross-encoder scores confidently as irrelevant.

Retrieval metrics: one list, three numbers

recall@k — what fraction of the relevant docs made the top k? · **MRR** — 1 / rank of the first relevant doc, averaged over queries · **nDCG@k** — graded relevance, discounted by log of position, normalised by the ideal ranking.



2 relevant docs exist for this query; nDCG normalises by the ideal order [2, 1]

Which metric for which stage

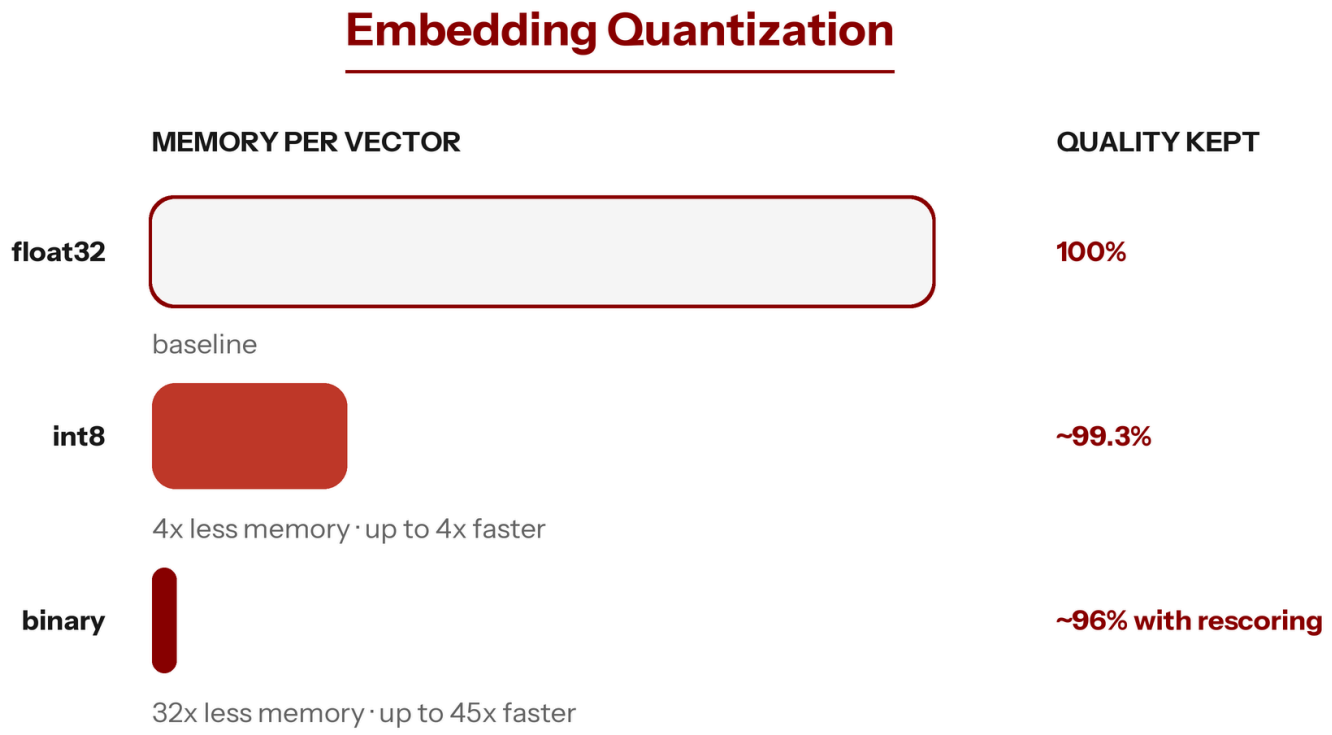
Metric	Question it answers	Stage
recall@k	What fraction of the right documents got through?	Stage one — candidate generation
MRR	How high is the *first* right answer?	"One right answer" tasks
nDCG@k	Is the whole top-k in the right order, with graded relevance?	Stage two — the reranker

WHAT THESE DO NOT MEASURE

Whether the *generated answer* is faithful and relevant. That is answer evaluation — Lecture 13.

Serving levers: smaller vectors

Quantize the stored vectors. int8: exactly 4× less memory. Binary: exactly 32× less — and a float rescoring pass recovers most of the quality.



Hugging Face benchmark (vendor-reported); rescoring re-ranks the binary top-k with float vectors

Serving levers: Matryoshka and the cascade

MATRYOSHKA EMBEDDINGS

- Trained so a **prefix** of the vector is itself a usable embedding
- Truncate dimensions for speed and memory
- Reported up to **14× smaller** at the same accuracy — on ImageNet; transfer to text models varies

THE CASCADE, AGAIN

- Cheap stages on many, expensive stages on few
- Truncated or binary vectors for the first pass
- Full vectors, then a reranker, on the survivors

> **Some levers trade quality for speed, some barely do — measure both.**

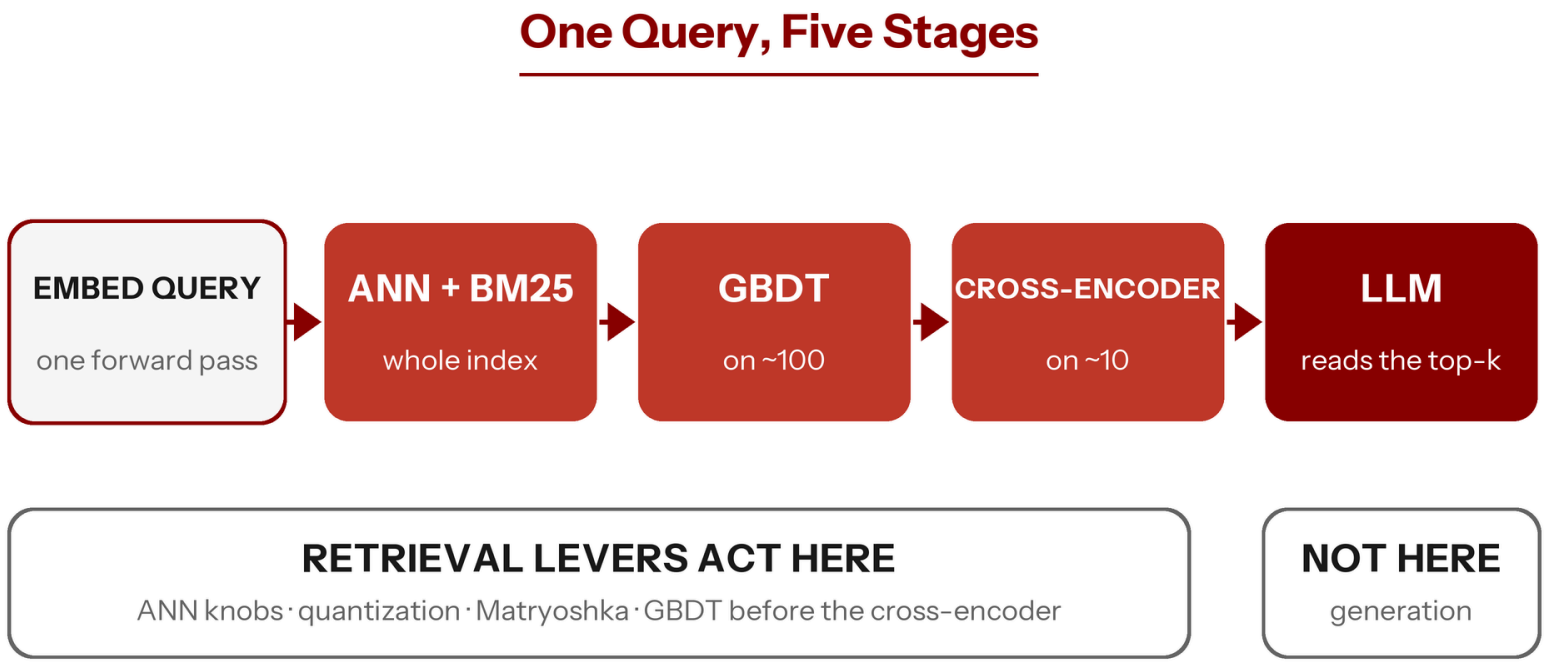
DISCUSSION

Suppose our Backrooms RAG is slow and sometimes wrong.

Which retrieval changes would you test — and what would each buy: latency, quality, or training cost?

Answer 1 — where the time goes

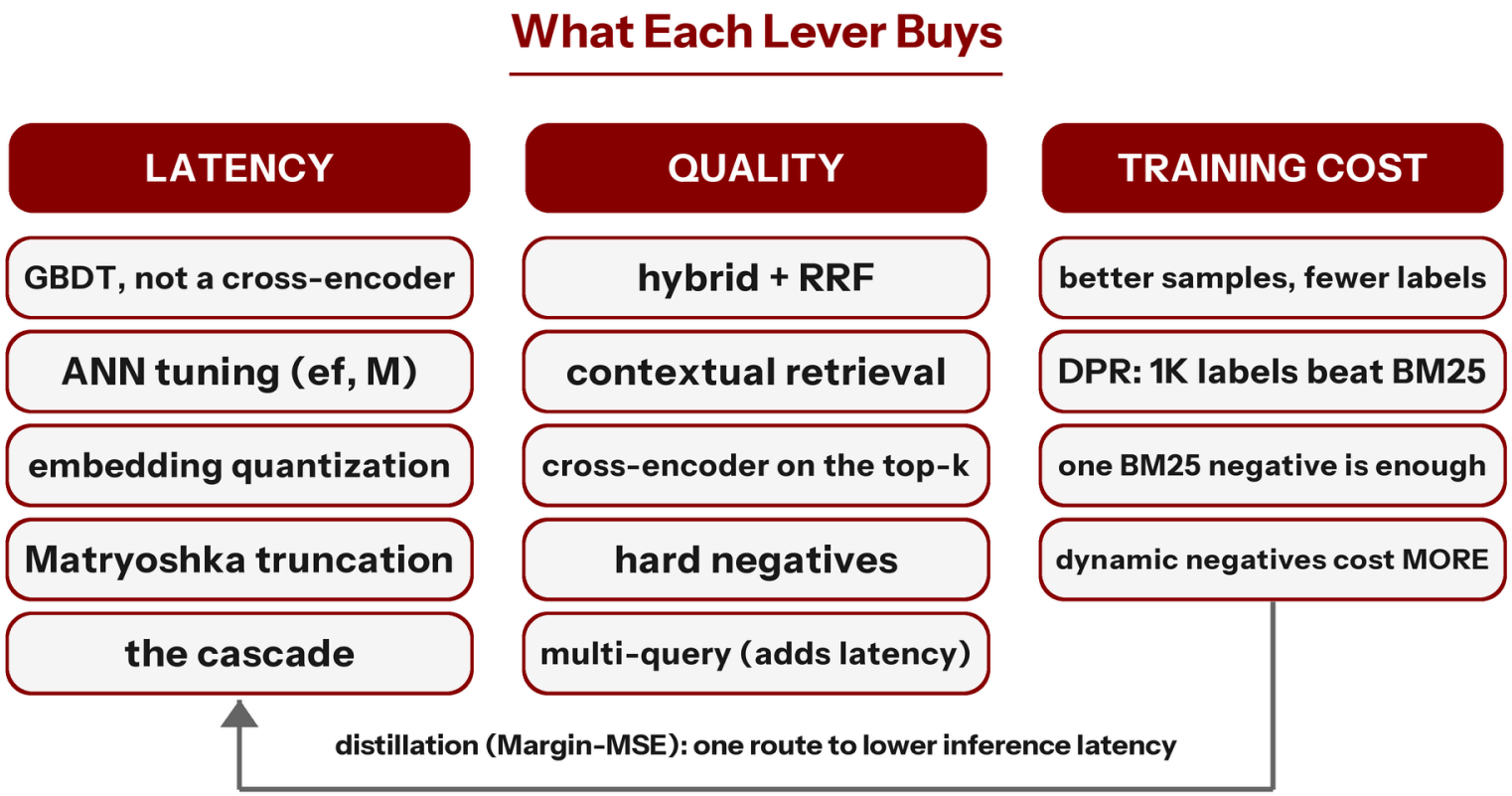
Every stage costs time, and the cheap ones run on the most documents. The levers you own act on retrieval — not on the generation call.



The seminar notebook measures each stage in ms per query on the same corpus

Answer 2 — what each lever buys

The naive answer — "better training data makes it faster" — is wrong: better samples buy **quality and fewer labels**, not milliseconds.



Summary

WHAT WE COVERED

- Why retrieval survives million-token windows
- Chunking, context expansion, embeddings, ANN
- Hybrid search with RRF
- The Backrooms RAG pipeline — and its limits
- Two-stage retrieval, GBDT vs cross-encoder
- Hard negatives, recall@k / MRR / nDCG, serving levers

WHAT TO RETAIN

- Search focused units; return enough context
- Keep BM25: hybrid + RRF
- Stage one for recall, stage two for precision
- Unlabelled is not negative — denoise
- Each lever buys latency, quality *or* fewer labels — know which

References — retrieval and long context

- Liu et al. 2023 · **Lost in the Middle: How Language Models Use Long Contexts** · arXiv 2307.03172
- Barnett et al. 2024 · **Seven Failure Points When Engineering a Retrieval Augmented Generation System** · arXiv 2401.05856
- Malkov & Yashunin 2016 · **Efficient and robust approximate nearest neighbor search using HNSW graphs** · arXiv 1603.09320
- Douze et al. 2024 · **The Faiss library** · arXiv 2401.08281
- Anthropic 2024 · **Introducing Contextual Retrieval** · anthropic.com/news/contextual-retrieval
- Cormack, Clarke & Büttcher 2009 · **Reciprocal Rank Fusion outperforms Condorcet and individual Rank Learning Methods** · SIGIR 2009
- Thakur et al. 2021 · **BEIR: A Heterogeneous Benchmark for Zero-shot Evaluation of IR Models** · arXiv 2104.08663
- Jimeno Yepes et al. 2024 · **Financial Report Chunking for Effective Retrieval Augmented Generation** · arXiv 2402.05131
- Xu et al. 2024 · **Retrieval-Augmented Generation with Knowledge Graphs for Customer Service Question Answering** · SIGIR 2024 · arXiv 2404.17723

References — deployments, data and ranking

- Jia et al. 2024 · **Path to high-quality LLM-based Dasher support automation** · DoorDash Engineering blog
- Wadden et al. 2020 · **Fact or Fiction: Verifying Scientific Claims** · arXiv 2004.14974
- Nogueira & Cho 2019 · **Passage Re-ranking with BERT** · arXiv 1901.04085
- Khattab & Zaharia 2020 · **ColBERT** · arXiv 2004.12832
- Burges 2010 · **From RankNet to LambdaRank to LambdaMART: An Overview** · MSR-TR-2010-82
- Haldar et al. 2018 · **Applying Deep Learning to Airbnb Search** · arXiv 1810.09591
- Karpukhin et al. 2020 · **Dense Passage Retrieval for Open-Domain QA** · arXiv 2004.04906
- Xiong et al. 2020 · **Approximate Nearest Neighbor Negative Contrastive Learning (ANCE)** · arXiv 2007.00808
- Qu et al. 2021 · **RocketQA** · arXiv 2010.08191

References — training, serving and tools

- Hofstätter et al. 2020 · **Improving Efficient Neural Ranking Models with Cross-Architecture Knowledge Distillation** · arXiv 2010.02666
- Kusupati et al. 2022 · **Matryoshka Representation Learning** · arXiv 2205.13147
- Järvelin & Kekäläinen 2002 · **Cumulated gain-based evaluation of IR techniques** · ACM TOIS 20(4)
- Hugging Face 2024 · **Embedding Quantization** · huggingface.co/blog/embedding-quantization
- Sentence-Transformers · **Retrieve & Re-Rank; Pretrained Cross-Encoders** · sbert.net
- hnswlib · **ALGO_PARAMS.md** · github.com/nmslib/hnswlib
- LightGBM · **Parameters (lambdarank, rank_xendcg)**; CatBoost · **Ranking loss functions (YetiRank)**
- LlamaIndex 0.14.25 · **SentenceSplitter; SemanticSplitter; AutoMergingRetriever; sentence windows** · developers.llamaindex.ai
- BAAI · **bge-small-en-v1.5 model card** · huggingface.co/BAAI/bge-small-en-v1.5
- English Backrooms Wiki · **50-article snapshot, 6 Nov 2026** · backrooms-wiki.wikidot.com · CC BY-SA 3.0; article credits in demo data/ATTRIBUTION.md

What's Next

LECTURE 07 — MEMORY IN AGENTS & AGENTIC RAG

Today you learned to retrieve. Next: what happens when an **agent** decides what to retrieve — and when the context itself outgrows the window.

Thank You

Questions & Discussion

Lecture 06 · LLM Agent Development